COT-4400: Analysis of Algorithms

Final Project Report

Graph Algorithms in Network Optimization: Shortest Path Routing in Internet Router Networks

Team Members

Seyoung Kan – U36444259

Kaushik Selvakumar – U75300364

USF Bellini College of AI, Cybersecurity and Computing

April 2026

Team Contribution Statement

Member	Student ID	Contributions
Seyoung Kan	U36444259	Problem formulation, graph design, BFS & DFS implementation, complexity analysis (BFS/DFS/Dijkstra), report writing (Sections 1-3)
Kaushik Selvakumar	U75300364	Dijkstra, Bellman-Ford & Kruskal implementation, experimental analysis, complexity analysis (BF/Kruskal), report writing (Sections 4-6), presentation

1. Introduction

The internet is one of the most complex systems we rely on today. Behind the scenes, it's just a massive network of routers constantly deciding how to send data from one place to another as efficiently as possible.

Every time data is sent, routers need to figure out the best path to take, usually trying to minimize delay. That's where graph algorithms come in.

In this project, we model a router network as a directed, weighted graph and apply core graph algorithms like BFS, DFS, Dijkstra, Bellman-Ford, and Kruskal's MST. Each one solves a different part of the problem, whether that's exploring the network, finding the shortest path, or optimizing the overall structure.

2. Problem Formulation & Graph Modeling

2.1 Real-World Problem

The main problem here is straightforward: given a network of routers, how do we send data from one router to another in the most efficient way?

In real networks, bad routing decisions can lead to higher latency, congestion, or even packet loss. So, this isn't just theoretical — this is exactly what real protocols like OSPF and BGP are solving.

2.2 Graph Representation

We model the router network as a graph:

- **Vertices (20 total):** each one represents a router (R0–R19)
- **Edges (33 total):** each one represents a directed connection between routers
- **Weights:** represent latency in milliseconds

So basically:

$$G = (V, E)$$

Where:

- The graph is **directed** (since routing isn't always symmetric)
- The graph is **weighted** (since links have different delays)
- And we assume **no negative weights** (since latency can't be negative)

2.3 Graph Architecture Layers

Layer	Routers	Role
Core	R0, R1	High-speed backbone; interconnect all distribution routers
Distribution	R2–R4	Aggregate edge-layer traffic; route to core
Edge	R5–R9	Connect to ISPs and CDNs; first-hop for external traffic
ISP/CDN	R10–R13	Internet service providers and content delivery nodes
Firewall	R14, R15	Security gateway before access subnets
Access	R16–R19	End-user subnets; final hop to user devices

3. Graph Construction & Dataset

We built a synthetic dataset with:

- **20 routers**
- **33 directed edges**

The weights were chosen to represent realistic latency differences across the network

3.1 Router Dataset

The table below summarizes all routers in the network, including their labels, layers, and roles within the system.

ID	Label	Layer	Role
R0	CoreA	Core	Internet backbone hub, connects to all distribution routers
R1	CoreB	Core	Redundant backbone hub
R2	DistA	Distribution	Aggregates EdgeA and EdgeB
R3	DistB	Distribution	Aggregates EdgeC and EdgeD
R4	DistC	Distribution	Aggregates EdgeB and EdgeE
R5	EdgeA	Edge	Connects to ISP1
R6	EdgeB	Edge	Connects to ISP2
R7	EdgeC	Edge	Connects to CDN1
R8	EdgeD	Edge	Connects to CDN2
R9	EdgeE	Edge	Connects to ISP1
R10	ISP1	ISP	Internet Service Provider 1
R11	ISP2	ISP	Internet Service Provider 2
R12	CDN1	CDN	Content Delivery Network 1
R13	CDN2	CDN	Content Delivery Network 2
R14	FW1	Firewall	Protects AccessA and AccessB subnets
R15	FW2	Firewall	Protects AccessC and AccessD
R16	AccessA	Access	End-user subnet (A)
R17	AccessB	Access	End-user subnet (B)
R18	AccessC	Access	End-user subnet (C)
R19	AccessD	Access	End-user subnet (D)

3.2 Key Edge Connections

Screenshot: SECTION 1: Graph Adjacency List (20 routers, 33 directed edges)

```
SECTION 1 — Graph Adjacency List

=== Graph Adjacency List ===
(Format: Router -> Neighbor [weight ms])

R0  -> R1[10ms], R2[15ms], R3[20ms]
R1  -> R4[12ms], R5[18ms]
R2  -> R4[8ms],  R6[14ms]
R3  -> R6[22ms], R7[9ms]
R4  -> R8[11ms], R9[7ms]
R5  -> R8[13ms], R10[6ms]
R6  -> R9[16ms], R11[10ms]
R7  -> R11[5ms], R12[20ms]
R8  -> R13[9ms], R14[14ms]
R9  -> R13[18ms], R15[11ms]
R10 -> R14[7ms], R15[13ms]
R11 -> R16[8ms], R17[15ms]
R12 -> R17[6ms], R18[19ms]
R13 -> R19[10ms]
R14 -> R19[12ms]
R15 -> R19[9ms]
R16 -> R19[7ms]
R17 -> R19[11ms]
R18 -> R19[14ms]
R19 -> (no outgoing edges)

Total Routers (Vertices): 20
Total Connections (Edges): 33
```

Source: Source: Terminal output after running main.cpp

Below are representative edges from the actual implementation used in the code

From	To	Weight (ms)	Type
R0	R1	10	Core → Edge
R0	R2	15	Core → Edge
R0	R3	20	Core → Edge
R1	R4	12	Edge → Distribution
R1	R5	18	Edge → Distribution
R2	R4	8	Edge → Distribution

R2	R6	14	Edge → Distribution
R3	R7	9	Edge → Distribution
R4	R9	7	Distribution → Access
R9	R15	11	Access → Gateway
R7	R11	5	Distribution → Access
R15	R19	9	Gateway link
R7	R11	5	Distribution → Access
R11	R16	8	Access → Gateway
R16	R19	7	Gateway Link
R13	R19	10	Gateway Link
R14	R19	12	Gateway Link

4. Algorithm Implementation

4.1 BFS – Breadth-First Search

BFS explores the graph level by level using a queue.

- Time: **O(V + E)**
- Space: **O(V)**

It's useful for:

- checking if nodes are reachable
- finding shortest paths in terms of number of hops

Screenshot: SECTION 2 — BFS Output

```
SECTION 2 — BFS Traversal from R0

Time Complexity:  O(V + E)
Space Complexity: O(V)

Visit Order:
  [1]  Visited R0
  [2]  Visited R1
  [3]  Visited R2
  [4]  Visited R3
  [5]  Visited R4
  [6]  Visited R5
  [7]  Visited R6
  [8]  Visited R7
  [9]  Visited R8
  [10] Visited R9
  [11] Visited R10
  [12] Visited R11
  [13] Visited R12
  [14] Visited R13
  [15] Visited R14
  [16] Visited R15
  [17] Visited R16
  [18] Visited R17
  [19] Visited R18
  [20] Visited R19

Routers unreachable from R0: None (all routers reachable)
```

Source: Terminal output after running main.cpp

Screenshot: SECTION 3 — DFS Output

```
SECTION 3 — DFS Traversal from R0

Time Complexity:  O(V + E)
Space Complexity: O(V)

Visit Order:
  [1]  Visited R0
  [2]  Visited R1
  [3]  Visited R4
  [4]  Visited R8
  [5]  Visited R13
  [6]  Visited R19
  [7]  Visited R14
  [8]  Visited R9
  [9]  Visited R15
  [10] Visited R5
  [11] Visited R10
  [12] Visited R2
  [13] Visited R6
  [14] Visited R11
  [15] Visited R16
  [16] Visited R17
  [17] Visited R3
  [18] Visited R7
  [19] Visited R12
  [20] Visited R18

DFS complete. Total routers visited: 20/20
```

Source: Terminal output after running main.cpp

BFS Pseudocode

```
BFS(Graph G, source s):
  dist[v] = -1  for all v in V        // -1 = unvisited
  dist[s] = 0
  queue Q = {s}
  while Q is not empty:
    u = dequeue(Q)
    for each neighbor v of u:
      if dist[v] == -1:
        dist[v] = dist[u] + 1
        parent[v] = u
        enqueue(Q, v)
```

4.2 DFS – Depth-First Search

DFS goes as deep as possible before backtracking.

- Time: **O(V + E)**
- Space: **O(V)**

It's mainly used for:

- cycle detection
- exploring structure of the graph

DFS Pseudocode

```
DFS(Graph G, source s):
  visited[v] = false  for all v in V
  DFS_Visit(s)

DFS_Visit(u):
  visited[u] = true
  for each neighbor v of u:
    if not visited[v]:
      parent[v] = u
      DFS_Visit(v)
```

4.3 Dijkstra's Algorithm

Dijkstra is used to find the shortest path based on weights (latency).

- Time: **O((V + E) log V)**
- Space: **O(V)**

It works great as long as there are no negative weights.

Example Shortest Path:

From router R0 to R19:

R0 → R1 → R4 → R9 → R15 → R19 = 49 ms

This matches the output generated by the program.

Screenshot: SECTION 4 — Dijkstra Output (R19 | 49ms | R0 → R1 → R4 → R9 → R15 → R19)

```
SECTION 4 — Dijkstra's Shortest Path from R0

NOTE: Does NOT support negative edge weights.
Time Complexity:  O((V + E) log V)
Space Complexity: O(V)

------------------------------------------------------------
Destination | Latency (ms) | Path
------------------------------------------------------------
R0          | 0ms          | R0
R1          | 10ms         | R0 -> R1
R2          | 15ms         | R0 -> R2
R3          | 20ms         | R0 -> R3
R4          | 22ms         | R0 -> R1 -> R4
R5          | 28ms         | R0 -> R1 -> R5
R6          | 29ms         | R0 -> R2 -> R6
R7          | 29ms         | R0 -> R3 -> R7
R8          | 33ms         | R0 -> R1 -> R4 -> R8
R9          | 29ms         | R0 -> R1 -> R4 -> R9
R10         | 34ms         | R0 -> R1 -> R5 -> R10
R11         | 34ms         | R0 -> R3 -> R7 -> R11
R12         | 49ms         | R0 -> R3 -> R7 -> R12
R13         | 42ms         | R0 -> R1 -> R4 -> R8 -> R13
R14         | 41ms         | R0 -> R1 -> R5 -> R10 -> R14
R15         | 40ms         | R0 -> R1 -> R4 -> R9 -> R15
R16         | 42ms         | R0 -> R3 -> R7 -> R11 -> R16
R17         | 49ms         | R0 -> R3 -> R7 -> R11 -> R17
R18         | 68ms         | R0 -> R3 -> R7 -> R12 -> R18
R19         | 49ms         | R0 -> R1 -> R4 -> R9 -> R15 -> R19  ★ SHORTEST PATH

------------------------------------------------------------
```

Source: Source: Terminal output after running main.cpp

Dijkstra Pseudocode

```
Dijkstra(Graph G, source s):
  dist[v] = ∞  for all v in V
  dist[s] = 0
  MinHeap Q = {(0, s)}
  while Q is not empty:
    (d, u) = extract_min(Q)
    if d > dist[u]: continue          // stale entry
```

```
    for each edge (u→v, weight w):
      if dist[u] + w < dist[v]:
        dist[v] = dist[u] + w
        parent[v] = u
        insert(Q, (dist[v], v))
```

4.4 Bellman-Ford Algorithm

Bellman-Ford is similar to Dijkstra but more flexible.

- Time: **O(V × E)**
- Space: **O(V)**

It can:

- handle negative weights
- detect negative cycles

But it’s slower, so it’s not ideal for large networks.

Screenshot: SECTION 5 — Bellman-Ford Output

```
SECTION 5 — Bellman-Ford Shortest Path from R0

Supports negative edge weights. Detects negative cycles.
Best Case Time:  O(E)     (paths stabilize in 1 pass)
Worst Case Time: O(V * E)
Space Complexity: O(V)

Early termination after pass 2 (Best Case achieved)

------------------------------------------------------------
Destination | Latency (ms) | Path
------------------------------------------------------------
R0          | 0ms          | R0
R1          | 10ms         | R0 -> R1
R2          | 15ms         | R0 -> R2
R3          | 20ms         | R0 -> R3
R4          | 22ms         | R0 -> R1 -> R4
R5          | 28ms         | R0 -> R1 -> R5
R6          | 29ms         | R0 -> R2 -> R6
R7          | 29ms         | R0 -> R3 -> R7
R8          | 33ms         | R0 -> R1 -> R4 -> R8
R9          | 29ms         | R0 -> R1 -> R4 -> R9
R10         | 34ms         | R0 -> R1 -> R5 -> R10
R11         | 34ms         | R0 -> R3 -> R7 -> R11
R12         | 49ms         | R0 -> R3 -> R7 -> R12
R13         | 42ms         | R0 -> R1 -> R4 -> R8 -> R13
R14         | 41ms         | R0 -> R1 -> R5 -> R10 -> R14
R15         | 40ms         | R0 -> R1 -> R4 -> R9 -> R15
R16         | 42ms         | R0 -> R3 -> R7 -> R11 -> R16
R17         | 49ms         | R0 -> R3 -> R7 -> R11 -> R17
R18         | 68ms         | R0 -> R3 -> R7 -> R12 -> R18
R19         | 49ms         | R0 -> R1 -> R4 -> R9 -> R15 -> R19
------------------------------------------------------------
```

Source: Terminal output after running main.cpp

Bellman-Ford Pseudocode

```
BellmanFord(Graph G, source s):
  dist[v] = ∞  for all v in V
  dist[s] = 0
  for i = 1 to V-1:
    for each edge (u→v, weight w) in E:
      if dist[u] + w < dist[v]:
        dist[v] = dist[u] + w
        parent[v] = u
  // Negative cycle detection (V-th relaxation)
  for each edge (u→v, weight w) in E:
    if dist[u] + w < dist[v]:
      report NEGATIVE CYCLE
```

4.5 Kruskal's MST Algorithm

Kruskal's algorithm builds a minimum spanning tree using Union-Find.

- Time: **O(E log E)**
- Space: **O(V)**

It's useful for:

- minimizing infrastructure cost
- building an efficient network backbone

Note: *Since MST is defined for undirected graphs, Kruskal's algorithm is applied to an* ***undirected interpretation of the router network.***

Screenshot: SECTION 6 — Kruskal MST Output (Total edges: 19, Total weight: 171ms)

```
SECTION 6 — Kruskal's Minimum Spanning Tree

NOTE: MST computed on undirected version of the graph.
Time Complexity:  O(E log E)
Space Complexity: O(V)

MST Edges (sorted by inclusion order):
--------------------------------------------------
Edge                | Weight (ms)
--------------------------------------------------
R7  -- R11          | 5ms
R5  -- R10          | 6ms
R12 -- R17          | 6ms
R4  -- R9           | 7ms
R10 -- R14          | 7ms
R16 -- R19          | 7ms
R2  -- R4           | 8ms
R11 -- R16          | 8ms
R3  -- R7           | 9ms
R8  -- R13          | 9ms
R15 -- R19          | 9ms
R0  -- R1           | 10ms
R6  -- R11          | 10ms
R13 -- R19          | 10ms
R4  -- R8           | 11ms
R17 -- R19          | 11ms
R1  -- R4           | 12ms
R14 -- R19          | 12ms
R18 -- R19          | 14ms
--------------------------------------------------
Total MST edges:  19
Total MST weight: 171ms
```

Source: Source: Terminal output after running main.cpp

Kruskal Pseudocode

```
KruskalMST(Graph G):
  Sort all edges by weight ascending
  Initialize DSU with V components
  MST = {}
  for each edge (u, v, w) in sorted order:
    if find(u) ≠ find(v):       // no cycle
      MST.add(u, v, w)
      union(u, v)
```

```
    if |MST| == V-1: break
  return MST
```

5. Complexity Analysis

Algorithm	Time	Space	Worst Case	Best Case
BFS	O(V + E)	O(V)	O(V+E) – dense	O(V) – tree
DFS	O(V + E)	O(V)	O(V+E) – dense	O(V) – tree
Dijkstra	O((V+E) log V)	O(V)	Dense graph	Linear graph
Bellman-Ford	O(V × E)	O(V)	O(V×E) – always	Early stop possible
Kruskal	O(E log E)	O(V)	Dense graph	Sparse graph

5.1 Detailed Explanations

BFS / DFS – O(V+E) time: Each vertex is visited once (O(V)), and each edge is examined once (O(E)). Space is O(V) for the queue/stack and visited array. Best case occurs on a tree (sparse); worst case on a dense graph.

Dijkstra – O((V+E) log V) time: Each vertex is extracted from the min-heap once (O(V log V)), and each edge may trigger a heap insertion (O(E log V)). With a Fibonacci heap this reduces to O(E + V log V). Space is O(V) for distances and parent arrays plus O(V+E) for the heap.

Bellman-Ford – O(V×E) time: The outer loop runs V-1 times, and each iteration scans all E edges. Space is O(V). Best case: early termination after one stable iteration. Worst case: always V-1 iterations (linear chain graph with negative edges).

Kruskal – O(E log E) time: Dominated by sorting the edge list. Union-Find operations are nearly O(1) amortized with path compression. Space is O(V) for the DSU structure. Best case: sparse graph with few edges; worst case: dense graph with E close to V^2.

6. Experimental Analysis & Comparison

Screenshot: SECTION 7 — Experimental Results Table

```
SECTION 7 — Experimental Performance Analysis

Timing unit: microseconds (µs)
Source node: 0 for all traversal/SSSP algorithms

Graph Sizes:
  Small Sparse: 10 nodes, 12 edges
  Small Dense:  10 nodes, 45 edges
  Large Sparse: 50 nodes, 65 edges
  Large Dense:  50 nodes, 1225 edges

--------------------------------------------------------------------------------
Algorithm     | Small Sparse    | Small Dense     | Large Sparse    | Large Dense
--------------------------------------------------------------------------------
BFS           | 0.561µs         | 0.231µs         | 0.260µs         | 1.644µs
DFS           | 0.250µs         | 0.180µs         | 0.361µs         | 1.623µs
Dijkstra      | 0.461µs         | 0.471µs         | 1.112µs         | 3.317µs
Bellman-Ford  | 0.170µs         | 0.140µs         | 0.611µs         | 1.644µs
Kruskal       | 0.541µs         | 1.172µs         | 2.194µs         | 50.466µs
--------------------------------------------------------------------------------

Key Observations:
  - BFS/DFS are fastest (simple traversal, O(V+E))
  - Dijkstra is efficient for sparse graphs (O((V+E) log V))
  - Bellman-Ford is slowest on dense graphs (O(V*E))
  - Kruskal scales with edges (O(E log E))
```

Source: results/experimental_results.md or terminal output

6.1 Execution Time Results

Algorithm	Complexity	Small Sparse	Small Dense	Large Sparse	Large Dense
BFS	O(V+E)	0.561 µs	0.231 µs	0.260 µs	1.644 µs
DFS	O(V+E)	0.250 µs	0.180 µs	0.361 µs	1.623 µs
Dijkstra	O((V+E)logV)	0.461 µs	0.471 µs	1.112 µs	3.317 µs

Bellman-Ford	O(V×E)	0.170 μs	0.140 μs	0.611 μs	1.644 μs
Kruskal	O(E log E)	0.541 μs	1.172 μs	2.194 μs	50.466 μs

Results were measured using C++

chrono::high_resolution_clock

on a 20-router directed graph with 33 edges. As expected, BFS and DFS are fastest due to their O(V+E) linear complexity.

- All algorithms perform efficiently at this scale, with BFS/DFS being fastest.
- Kruskal shows noticeable growth on dense graphs due to sorting overhead.
- Differences between Dijkstra and Bellman-Ford are minimal at this graph size.

6.2 Sparse vs Dense Graph Comparison

Our 20-router graph is moderately sparse (33 edges vs. a maximum of 380 for a complete directed graph).

For experimental analysis, larger graphs were generated in both sparse and dense configurations. The dense case approaches complete graph behavior, where the number of edges grows significantly relative to nodes.

On sparse graphs, Dijkstra performs significantly better than Bellman-Ford since fewer edge relaxations are required. On dense graphs, Bellman-Ford becomes much slower due to its O(V × E) complexity, while Dijkstra remains more efficient with O((V + E) log V).

Observations

- BFS and DFS scale linearly with O(V + E) and show minimal variation across densities
- Dijkstra scales much better than Bellman-Ford as graph size increases
- Bellman-Ford becomes significantly slower on dense graphs
- Kruskal’s performance depends on the number of edges and grows with O(E log E)

6.3 Graph Size Variation

For small graphs (V=10, E~15), all algorithms complete in under 1 μs — negligible differences. For medium graphs (V=100, E~300), Dijkstra's advantage becomes measurable (~50 μs vs ~200 μs for Bellman-Ford). For large graphs (V=1000, E~5000), Bellman-Ford can take seconds while Dijkstra completes in milliseconds. BFS/DFS remain fastest at all scales.

7. Discussion & Insights

7.1 Best Performing Algorithm

For this specific problem — finding minimum-latency paths in a non-negative weighted internet topology — Dijkstra's algorithm performed best. It produces optimal shortest paths in O((V+E) log V) time, which is significantly more efficient than Bellman-Ford's O(V×E) for large networks. Real-world protocols like OSPF and IS-IS implement Dijkstra for exactly this reason. Dijkstra is clearly the best choice for shortest path when all weights are non-negative.

7.2 When to Use Each Algorithm

- BFS: Use when all edges have equal weight or when minimizing hop count. Also for level-order traversal and checking graph connectivity **(reachability / hop count)**.
- DFS: Use for topology discovery, cycle detection, topological sorting, and finding strongly connected components (Tarjan's, Kosaraju's) **(structure / cycles)**.
- Dijkstra: Use for shortest-path problems with non-negative weights. Ideal for large-scale routing protocols (OSPF, GPS navigation, game pathfinding) **(shortest path (most cases))**.
- Bellman-Ford: Use when negative edge weights are possible (e.g., currency arbitrage, network cost models with subsidies). Also mandatory for detecting negative-weight cycles **(when negatives exist)**.
- Kruskal's MST: Use for network infrastructure planning — finding the minimum-cost set of links to connect all routers without cycles **(network design)**.

7.3 Algorithm Limitations

- Dijkstra fails with negative-weight edges — it assumes once a node is finalized, its distance is optimal, which breaks with negative edges.
- Bellman-Ford is too slow (O(V×E)) for large-scale real-time routing. The internet has millions of routers; BF would be impractical.
- Kruskal's MST produces an undirected tree — in directed networks, we would need a Minimum Spanning Arborescence (Edmonds' algorithm).
- BFS/DFS ignore edge weights — they cannot find minimum-latency paths, only minimum-hop paths.

7.4 Key Observations

A notable finding from our experiment was that Dijkstra and Bellman-Ford produced identical shortest-path distances on our dataset, confirming correctness of both implementations.

The shortest path from R0 to R19 is: **R0 → R1 → R4 → R9 → R15 → R19 = 49 ms**

The Minimum Spanning Tree contains **19 edges with a total weight of 171 ms**, representing the minimum-cost backbone required to connect all routers.

This highlights the difference between the two optimization goals:

- Shortest path minimizes latency for a single route
- MST minimizes total infrastructure cost across the entire network

8. Conclusion

This project demonstrated the application of five fundamental graph algorithms to a realistic internet router network. We modeled a 20-router, 33-edge directed weighted graph representing a hierarchical ISP topology and applied BFS, DFS, Dijkstra, Bellman-Ford, and Kruskal's MST algorithms.

Each algorithm plays a different role:

- BFS/DFS for exploring the network
- Dijkstra for routing decisions
- Bellman-Ford for edge cases
- Kruskal for designing the network itself

Overall, this gives a good picture of how theoretical algorithms are actually used in real systems like the internet. Future work could extend this model to include dynamic link failures, traffic-aware weights, and comparison with the A* algorithm for destination-directed routing.